

Unit IV: Clustering - Exam Preparation Booklet

Table of Contents

1. Introduction to Clustering
2. Hierarchical Clustering (AHC & DHC)
3. Centroid-based Clustering (K-Means, K-Medoids, K-Prototypes)
4. Density-based Clustering (DBSCAN)
5. Distribution-based Clustering (GMM)
6. Applications of Clustering

1. Introduction

What is Clustering?

Clustering is a technique in **Unsupervised Learning** where the goal is to group a set of objects in such a way that objects in the same group (cluster) are more similar to each other than to those in other groups (clusters).

Unlike Supervised Learning, there are **no predefined labels**. The algorithm must find the structure in the data on its own.

Need of Clustering

We use clustering for several critical reasons:

- **Exploratory Data Analysis (EDA):** To understand the underlying structure of a dataset before applying other models.
- **Pattern Recognition:** Identifying groups with similar behaviours (e.g., customers who buy similar items).
- **Dimensionality Reduction:** Reducing a large dataset into a few representative clusters.
- **Outlier Detection:** Points that do not belong to any cluster are often anomalies or noise.

Types of Clustering

1. **Connectivity-based (Hierarchical):** Based on the idea that objects are more related to nearby objects than to objects farther away (e.g., AHC).
2. **Centroid-based (Partitioning):** Represents clusters by a central vector (e.g., K-Means).
3. **Density-based:** Defines clusters as connected dense regions (e.g., DBSCAN).
4. **Distribution-based:** Assumes data is generated by a statistical distribution (e.g., Gaussian Mixture Models).

2. Hierarchical Clustering (Connectivity-based)

Hierarchical clustering builds a hierarchy of clusters. The result is usually presented as a tree-based representation called a **Dendrogram**.

A. Agglomerative Hierarchical Clustering (AHC)

This is a "**Bottom-Up**" approach.

How it works:

1. **Start:** Treat every data point as a single cluster. (If you have N points, you have N clusters).
2. **Calculate Distances:** Compute the distance (similarity) between all pairs of clusters.
3. **Merge:** Find the two closest clusters and merge them into a single cluster.
4. **Update:** Recalculate the distance between the new cluster and existing clusters.
5. **Repeat:** Repeat steps 3 and 4 until all data points are merged into a single large cluster.

Linkage Criteria (How to measure distance between clusters):

- *Single Linkage:* Distance between the closest points of two clusters.
- *Complete Linkage:* Distance between the farthest points of two clusters.
- *Average Linkage:* Average distance between all points.

B. Divisive Hierarchical Clustering (DHC)

This is a "Top-Down" approach.

How it works:

1. **Start:** Treat all data points as one single big cluster.
2. **Split:** Use a flat clustering method (like K-Means) to split the cluster into two dissimilar sub-clusters.
3. **Recursion:** Recursively apply this splitting logic to each sub-cluster.
4. **Stop:** Continue until each cluster contains only one data point (singleton).

3. Centroid-based / Partitioning Algorithms

These algorithms break the dataset into k distinct, non-overlapping subgroups.

A. K-Means Clustering Algorithm

K-Means is an iterative algorithm that partitions the dataset into K pre-defined distinct non-overlapping subgroups (clusters).

Steps:

1. **Initialization:** Choose K random points as initial cluster centroids.
2. **Assignment:** Assign each data point to the closest centroid (usually based on Euclidean distance).
3. **Update:** Calculate the new centroid of each cluster by taking the mean of all data points assigned to that cluster.
4. **Repeat:** Repeat steps 2 and 3 until the centroids stop changing (convergence) or a maximum number of iterations is reached.

Advantages:

- Simple to understand and implement.
- Computationally efficient (fast) for large datasets.
- Guarantees convergence.

Disadvantages:

- You must define the number of clusters (K) manually.
- Sensitive to initial centroid placement (can get stuck in local optima).
- **Sensitive to Outliers:** One outlier can drag the centroid significantly.
- Assumes clusters are spherical; struggles with complex shapes.

B. Finding the Optimal 'K'

1. The Elbow Method

- Run K-Means for a range of K values (e.g., 1 to 10).

- Calculate the **Within-Cluster Sum of Squares (WCSS)** for each K (sum of squared distances between points and their centroid).
- Plot WCSS against K .
- **Interpretation:** The WCSS decreases as K increases. The "Elbow" is the point where the rate of decrease shifts abruptly. This is the optimal K .

2. The Silhouette Method

- Measures how similar a point is to its own cluster (cohesion) compared to other clusters (separation).
- The score ranges from **-1 to +1**.
 - **+1:** Perfectly clustered.
 - **0:** Point is on the boundary.
 - **-1:** Point is assigned to the wrong cluster.
- The optimal K is the one that maximizes the average Silhouette score.

C. K-Medoids

K-Medoids is similar to K-Means, but instead of using the *mean* (a calculated average point) as the center, it uses an **actual data point** from the cluster, called the **Medoid**.

- **Algorithm:** Partitioning Around Medoids (PAM).
- **Key Difference:** It minimizes the sum of dissimilarities rather than Euclidean distances.
- **Advantage:** It is much **more robust to outliers** than K-Means because medoids are existing data points and not influenced by extreme values as easily as a mean.

D. K-Prototype

K-Means works on numerical data. K-Medoids works on numerical data. **K-Prototype** is designed for **Mixed Data**.

- It combines K-Means (for numeric features) and K-Modes (for categorical features).
- It uses a specific distance metric that sums Euclidean distance for the numeric attributes and a matching dissimilarity measure for categorical attributes.

4. Density-based Clustering

DBSCAN Algorithm

DBSCAN stands for **Density-Based Spatial Clustering of Applications with Noise**.

It groups together points that are closely packed together (points with many neighbours) and marks points as outliers if they lie alone in low-density regions.

Key Parameters:

1. **Epsilon (ϵ):** The radius of the circle around a data point.
2. **MinPts:** The minimum number of data points required inside the ϵ -radius to form a dense region.

How it works (Classification of Points):

1. **Core Point:** A point is a Core point if it has at least *MinPts* within its ϵ -radius.
2. **Border Point:** A point that has fewer than *MinPts* within its radius but is in the neighbourhood of a Core point.
3. **Noise Point (Outlier):** A point that is neither a Core point nor a Border point.

Algorithm Steps:

1. Pick an unvisited point.
2. If it is a Core Point, a cluster is formed. Recursively visit all density-connected points to expand the cluster.

3. If it is a Border Point, assign it to the core point's cluster.
4. If it is a Noise Point, ignore it (for now).
5. Repeat until all points are visited.

Advantages:

- Does **not** require specifying the number of clusters (K) beforehand.
- Can find clusters of **arbitrary shapes** (e.g., one cluster surrounding another like a ring).
- Robust to **outliers** (identifies them as noise).

Disadvantages:

- Struggles with datasets that have **varying densities** (different clusters needing different ϵ).
- Performance drops in very high-dimensional data (Curse of Dimensionality).

5. Distribution-based Clustering

Gaussian Mixture Model (GMM)

GMM is a probabilistic model. Instead of hard assignment (a point belongs to Cluster A), it uses **Soft Clustering** (a point has a 70% probability of belonging to Cluster A and 30% to Cluster B).

Concept:

- It assumes that the data is generated from a mixture of a finite number of **Gaussian distributions** with unknown parameters.
- Each cluster is represented by a Gaussian distribution (defined by its Mean μ and Variance σ^2).

How it works (EM Algorithm):

GMM uses the Expectation-Maximization (EM) algorithm:

1. **Expectation (E-step):** Calculate the probability that each data point belongs to each cluster.
2. **Maximization (M-step):** Update the parameters (μ , σ) of the Gaussians to maximize the likelihood of the data given the probabilities found in the E-step.
3. Repeat until convergence.

6. Application of Clustering Techniques

1. Market Segmentation:

- grouping customers based on purchasing history, demographics, or interests. This allows businesses to target specific ads to specific groups (e.g., "Budget Shoppers" vs. "Luxury Buyers").

2. Statistical Data Analysis:

- Used in social sciences and biology to group similar surveys or genetic sequences. It simplifies complex data into manageable groups for analysis.

3. Social Network Analysis:

- Identifying "communities" within a social network. For example, finding groups of friends on Facebook or professional circles on LinkedIn based on connection patterns.

4. Image Segmentation:

- Dividing a digital image into multiple segments (sets of pixels). For example, separating the foreground object from the background in a photo, or identifying tumors in medical X-rays.

5. Anomaly Detection:

- Any data point that does not fit well into a cluster is considered an anomaly. This is used

in **Credit Card Fraud Detection** (unusual spending patterns) or network security (intrusion detection).

Unit 5: Ensemble Learning Exam Booklet

1. Explain the following with respect to Ensemble Learning.

i) Need for Ensemble Learning

No single machine learning model is perfect. A single model (like a Decision Tree or Logistic Regression) may suffer from specific weaknesses depending on the data. The primary needs for ensemble learning are:

- **Bias-Variance Tradeoff:** Single models often struggle to balance bias (underfitting) and variance (overfitting). Ensembles can reduce variance (Bagging) or bias (Boosting).
- **Robustness:** A single model might be sensitive to noise or outliers in the training data. Ensembles average out these fluctuations, making the final prediction more stable.
- **Performance:** In almost all machine learning competitions (like Kaggle), ensemble methods consistently outperform single best models by capturing different patterns in the data.

ii) Advantages of Ensemble Methods

- **Improved Accuracy:** Aggregating predictions usually results in lower error rates than any individual contributing model.
- **Overfitting Reduction:** Techniques like Bagging help prevent the model from memorizing the training data, leading to better generalization on unseen data.
- **Handling Linear/Non-Linear Data:** By combining different types of models, ensembles can model complex boundaries that a single linear or simple non-linear model cannot capture.

iii) Ensemble Learning Limitations

- **Computational Cost:** Training multiple models requires significantly more time, memory, and computational power than training a single model.
- **Interpretability (Black Box Nature):** While a single Decision Tree is easy to interpret, a Random Forest (with 100 trees) is difficult to visualize or explain to stakeholders.
- **Complexity:** Setting up, tuning hyperparameters, and maintaining an ensemble pipeline is more complex than managing a single model.

2. What is Ensemble Learning? Explain any two ensemble learning techniques.

Definition:

Ensemble Learning is a machine learning paradigm where multiple models (often called "weak learners" or "base learners") are trained to solve the same problem and combined to get better results. The main hypothesis is that by combining multiple weak models, we produce a single "strong learner".

Technique A: Bagging (Bootstrap Aggregating)

- **Concept:** Bagging involves training multiple base learners in parallel on different subsets of the training data.
- **Mechanism:** It uses **Bootstrapping** (random sampling with replacement) to create dataset subsets. Each model votes, and the final prediction is an average (regression) or majority vote (classification).

- **Goal:** Primarily reduces **Variance**.
- **Example:** Random Forest.

Technique B: Boosting

- **Concept:** Boosting involves training base learners sequentially. Each new model attempts to correct the errors made by the previous models.
- **Mechanism:** It assigns higher weights to data points that were misclassified by previous models. The final prediction is a weighted sum of all models.
- **Goal:** Primarily reduces **Bias**.
- **Example:** AdaBoost, Gradient Boosting.

3. Compare Homogeneous and Heterogeneous ensemble methods.

Feature	Homogeneous Ensembles	Heterogeneous Ensembles
Base Learners	Uses the same type of algorithm for all base learners (e.g., 100 Decision Trees).	Uses different types of algorithms (e.g., a Decision Tree + SVM + Neural Network).
Logic	Relies on data sampling or sequential error correction to create diversity.	Relies on the unique strengths/mathematical properties of different algorithms to create diversity.
Complexity	Generally easier to implement and tune (fewer unique hyperparameters).	More complex to tune as each algorithm requires specific preprocessing and hyperparameter optimization.
Examples	Random Forest, AdaBoost, Gradient Boosting.	Stacking (Stacked Generalization), Voting Classifier.

4 & 5. Explain different types of voting mechanisms in ensemble learning.

1. Hard Voting (Majority Voting)

- **Mechanism:** Each base model predicts a class label. The ensemble counts the votes, and the class with the most votes becomes the final prediction.
- **Example:**
 - Model A: Class 1
 - Model B: Class 1
 - Model C: Class 0
 - **Result:** Class 1 (2 votes vs 1 vote).
- **Best for:** Classification problems where models output strict labels.

2. Soft Voting (Average Probability)

- **Mechanism:** Instead of distinct labels, the ensemble averages the **probability** scores predicted by each model for every class. The class with the highest average probability is selected.
- **Example:**
 - Model A: 90% sure it's Class 1.
 - Model B: 60% sure it's Class 1.
 - Model C: 40% sure it's Class 1.
 - Average: $(0.9 + 0.6 + 0.4) / 3 = 0.63$.
 - **Result:** Class 1 (63% probability).
- **Best for:** When models are well-calibrated; usually performs better than hard voting.

3. Weighted Voting

- **Mechanism:** Not all models are equal. Some models might have higher accuracy than others. In weighted voting, predictions from better models count more towards the final decision.

- **Formula:** $\hat{y} = \text{argmax} \sum_{i=1}^N w_i \times P_i(y)$ where w_i is the weight of model i .
- **Best for:** Scenarios where there is a high variance in the performance of base learners.

6. Differentiate the Bagging and Boosting approach.

Feature	Bagging (Bootstrap Aggregating)	Boosting
Training Style	Parallel: Models are trained independently at the same time.	Sequential: Models are trained one after another.
Data Sampling	Random sampling with replacement (Bootstrapping).	Reweighting: Hard-to-classify points get higher weights.
Error Focus	Reduces Variance (overfitting).	Reduces Bias (underfitting).
Model Dependency	Base learners are independent of each other.	Each new learner is dependent on the previous one's errors.
Overfitting Risk	Low (tends to help prevent it).	High (if the number of iterations is too large).
Example	Random Forest.	AdaBoost, XGBoost.

7. Explain AdaBoost Algorithm in detail.

AdaBoost (Adaptive Boosting) is one of the earliest and most popular boosting algorithms. It combines multiple "weak learners" (usually Decision Stumps—trees with only one split) to create a strong classifier.

How it works (Step-by-Step):

1. **Initialize Weights:** Assign equal weights $w_i = \frac{1}{N}$ to all N training samples.
2. **Iterative Training:** For $t = 1$ to T (number of iterations):
 - Train a weak learner $h_t(x)$ using the weighted samples.
 - Calculate the **Total Error** (ϵ_t) of the model. This is the sum of weights of the misclassified points.
 - Calculate Amount of Say (α_t): Determine how important this stump is. $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
 - Update Weights: Increase weights for misclassified points (so the next model focuses on them) and decrease weights for correctly classified points. $w_{\text{new}} = w_{\text{old}} \times e^{\pm \alpha_t}$
 - Normalize weights so they sum to 1.
3. **Final Prediction:** The final hypothesis $H(x)$ is a weighted sum of the weak learners: $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

Key Takeaway: AdaBoost learns by punishing mistakes. It forces subsequent models to focus on the "hard" examples.

8. Explain Gradient Boosting ensemble learning.

Gradient Boosting is a generalization of boosting to arbitrary differentiable loss functions. While AdaBoost changes sample weights, Gradient Boosting trains models on the **residual errors** of the previous model.

Core Concept:

Instead of guessing the label directly, each new tree tries to predict the error (residual) left behind by the previous tree.

The Algorithm Flow:

1. **Base Model:** Make an initial prediction $F_0(x)$ (usually the average of the target values for regression).
2. **Iterative Process:** For $m = 1$ to M :

- Calculate the Pseudo-residuals (r_{im}): The difference between the observed value and the current prediction.

$$r_{im} = y_i - F_{m-1}(x_i)$$
 - Fit a new weak learner (usually a decision tree) $h_m(x)$ to these residuals.
 - Find a learning rate γ (to prevent overfitting).
 - Update the Model: Add the new tree's prediction to the ensemble.

$$F_m(x) = F_{m-1}(x) + \gamma h_m(x)$$
3. **Final Output:** The sum of the initial prediction plus the weighted contributions of all trees.

9. Explain Random Forest ensembles with an example.

Random Forest is a **Bagging** technique that uses Decision Trees as base learners. Ideally, decision trees are prone to overfitting. Random Forest solves this by creating many trees and averaging them.

Two Key Randomness Factors:

1. **Bagging (Row Randomness):** Each tree is trained on a random subset of data (with replacement).
2. **Feature Randomness (Column Randomness):** When splitting a node in a tree, the algorithm considers only a random subset of features (e.g., $\sqrt{\text{total_features}}$) rather than all features. This ensures the trees are **uncorrelated**.

Example:

Imagine you want to predict if a movie will be a "Hit".

- **Tree 1** looks at (Actor, Budget) $\rightarrow$ Predicts "Hit".
- **Tree 2** looks at (Genre, Duration) $\rightarrow$ Predicts "Flop".
- **Tree 3** looks at (Director, Release Date) $\rightarrow$ Predicts "Hit".

Final Prediction: Majority Vote $\rightarrow$ "Hit".

By forcing trees to look at different features, Random Forest ensures that one dominant feature (e.g., a famous Actor) doesn't dictate every single tree, making the model robust.

10. Elaborate stacking approach of ensemble with example.

Stacking (Stacked Generalization) is a **Heterogeneous** ensemble method. Unlike Bagging/Boosting which combines models simply (voting/averaging), Stacking uses a "Meta-Learner" to learn *how* to combine the base models optimally.

Architecture:

- **Level 0 (Base Learners):** A diverse set of models (e.g., KNN, SVM, Decision Tree) that make predictions on the dataset.
- **Level 1 (Meta Learner):** A final model (often Logistic Regression or Linear Regression) that takes the *predictions* of the Level 0 models as input features to make the final prediction.

Example Scenario:

- **Dataset:** House Price Prediction.
- **Step 1 (Level 0):**
 - Model A (KNN) predicts: \$200k
 - Model B (Random Forest) predicts: \$210k
 - Model C (Neural Net) predicts: \$250k
- **Step 2 (New Dataset):** Create a new dataset where the input features are \$[200k, 210k, 250k]\$ and the target is the actual price.
- **Step 3 (Level 1):** Train a Meta-Learner (Linear Regression) on this new data. The Meta-Learner might learn that the Random Forest is usually right, but the Neural Net overestimates, and it assigns weights accordingly to output the final price.

11. XGBoost (Extreme Gradient Boosting)

Definition:

XGBoost stands for eXtreme Gradient Boosting. It is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. While standard Gradient Boosting trains models sequentially to correct errors, XGBoost improves upon this framework with system optimization and algorithmic enhancements. It is famous for dominating machine learning competitions (like Kaggle).

Key Features & Advantages:

1. **Regularization (The Key Difference):**
Standard Gradient Boosting is prone to overfitting. XGBoost includes L1 (Lasso) and L2 (Ridge) regularization terms in its objective function. This controls the complexity of the trees, preventing overfitting.
 - o Formula: The Objective Function in XGBoost is:
$$\text{\$Obj} = \sum L(y_i, \hat{y}_i) + \sum \Omega(f_k)$$

Where L is the loss function (error) and Ω is the regularization term (complexity penalty).
2. **Parallel Processing:**
Although Boosting is a sequential process (Tree 2 needs Tree 1), XGBoost parallelizes the node building process. It pre-sorts data features and uses multiple CPU cores to determine the best "split" for a tree node much faster than standard sklearn GBM.
3. **Handling Missing Values (Sparsity Awareness):**
XGBoost has a built-in routine to handle missing values. It learns the best direction (left or right node) to send missing values during training to minimize loss. You do not need to impute data manually before feeding it to XGBoost.
4. **Tree Pruning:**
Standard Gradient Boosting stops splitting a node when it encounters a negative loss (greedy approach). XGBoost grows the tree to its maximum depth (max_depth) and then prunes backward, removing splits that do not result in a positive gain.

Why use XGBoost in an exam context?

- **Speed:** Much faster than standard Gradient Boosting.
- **Performance:** Generally provides higher accuracy.
- **Flexibility:** Supports regression, classification, and ranking problems.

12. Stacking: Variance Reduction and Blending

Stacking (Stacked Generalization) is an ensemble technique that combines multiple classification or regression models via a **meta-classifier** or **meta-regressor**. The base level models (Level 0) are trained based on a complete training set, then the meta-model (Level 1) is trained on the outputs of the base level models.

A. Variance Reduction Learning in Stacking

While Bagging is the primary technique known for variance reduction, **Stacking achieves variance reduction through diversity**.

- **The Concept:** If you stack multiple complex models (e.g., a deep Decision Tree, a Neural Network, and a KNN), each model individually might have high variance (they fit the training data too closely and differently).
- **How Stacking Reduces Variance:**
 - o Different models make different mistakes on different parts of the data.
 - o By combining them using a Meta-Learner (Level 1), the ensemble "smooths out" these individual fluctuations.
 - o The Meta-Learner learns which base model is reliable for which type of data point.
 - o **Result:** The final prediction is more stable (lower variance) and usually more

accurate (lower bias) than any single base model.

B. Blending Learning

Blending is a specific variation of Stacking. While they share the same goal (combining base models), they differ significantly in **how the training data is used**.

Difference between Stacking and Blending:

1. **Standard Stacking:** Uses **Cross-Validation (Out-of-Fold predictions)**.
 - The data is split into K-folds.
 - Base models predict on the "held-out" fold.
 - These predictions are stacked to train the meta-learner.
 - *Pros:* Uses all data for training. *Cons:* Computationally expensive and complex to code.
2. **Blending:** Uses a **Hold-out Validation Set**.
 - **Step 1:** Split the training data into two sets: Train_Set (e.g., 70%) and Validation_Set (e.g., 30%).
 - **Step 2:** Train Base Models (Level 0) strictly on the Train_Set.
 - **Step 3:** Use these Base Models to make predictions on the Validation_Set.
 - **Step 4:** The Validation_Set predictions become the "features" to train the Meta-Learner (Level 1).
 - **Step 5:** The Meta-Learner is trained to predict the actual target values of the Validation_Set.

Advantages of Blending:

- **Simplicity:** Much easier to implement than the complex loop structure of Stacking.
- **No Data Leakage:** Because the validation set was never seen by the base models during their training, the meta-learner is robust against leakage.

Disadvantages of Blending:

- **Data Efficiency:** You use less data to train your base models (because you had to hold out 30% for the blending step). This is bad if your dataset is small.

Reinforcement Learning

Unit VI: Reinforcement Learning - Exam Preparation Booklet

Table of Contents

1. Introduction to Reinforcement Learning (RL)
2. Key Elements & Comparison (Supervised vs Unsupervised vs RL)
3. Markov Decision Process (MDP) & Bellman Equation
4. Q-Learning (Table, Function, & Algorithm)

1. Reinforcement Learning (RL)

What is Reinforcement Learning?

Reinforcement Learning is a type of Machine Learning where an **agent** learns how to behave in an **environment** by performing actions and seeing the results. Instead of being told what to do (like in Supervised Learning), the agent learns through **trial and error**. It receives feedback in the form of **Rewards** (for good actions) or **Penalties** (for bad actions). The goal is to maximize the total cumulative reward over time.

Need for Reinforcement Learning

We need RL for scenarios where:

1. **No Pre-defined Data:** There is no labelled dataset available to train a model.
2. **Sequential Decision Making:** The decision made now affects the future (e.g., a chess move affects the board state 10 turns later).
3. **Interaction:** The system must interact with a dynamic environment (e.g., a robot walking on uneven terrain).
4. **Delayed Rewards:** The outcome (success/failure) is not known immediately, but only after a sequence of steps.

2. Types, Elements, and Comparison

Supervised vs. Unsupervised vs. Reinforcement Learning

Feature	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Data	Labelled data (Input-Output pairs).	Unlabelled data.	No data initially; learns from interaction (Experience).
Feedback	Direct feedback (Correct/Incorrect).	No feedback.	Delayed feedback (Reward/Penalty).
Goal	Predict outcome/class.	Find hidden structure/clusters.	Maximize long-term reward.
Method	Mapping inputs to outputs.	Pattern recognition.	Trial and error (Exploration vs Exploitation).
Example	Spam detection.	Customer segmentation.	Learning to play Mario/Chess.

Types of Reinforcement Learning

1. **Positive Reinforcement:** Adding a positive stimulus (reward) after a behaviour to increase the likelihood of that behaviour happening again (e.g., giving a dog a treat for sitting).
2. **Negative Reinforcement:** Removing a negative stimulus to strengthen a behaviour (e.g., turning off a loud alarm when you wake up).

Elements of Reinforcement Learning

To describe an RL problem, we need these key components:

1. **Agent:** The learner or decision-maker (e.g., the robot).
2. **Environment:** The world the agent interacts with (e.g., the maze).
3. **State (\$\$S\$):** The current situation or position of the agent.
4. **Action (\$A\$):** The set of moves the agent can make.
5. **Reward (\$R\$):** The immediate feedback received after an action.
6. **Policy (\$\pi\$):** The strategy the agent uses to decide which action to take in a specific state.

Real-Time Applications

- **Robotics:** Robots learning to walk or grasp objects without hard-coded instructions.
- **Game Playing:** AlphaGo (Chess/Go) or bots playing video games (Dota 2, Mario).
- **Self-Driving Cars:** Learning to navigate traffic, change lanes, and park.
- **Industrial Automation:** Optimizing cooling systems in data centers (Google DeepMind).

Markov Property

- A system has the Markov Property when **the next state depends only on the current state**, not the full past.
- This makes the system **memoryless** — no history is needed if the current state is well-defined.
- In learning systems, this keeps models simple: the agent only needs S_t to predict S_{t+1} .
- If the environment breaks this rule, the agent's learning becomes unreliable because hidden past information affects decisions.
- You can fix violations by **adding more features** or using memory-based models (RNNs, Transformers).
- This property is the foundation for **Markov chains, MRP, and MDP**.

Markov Chain / Markov Process

- A Markov Chain is a sequence of states where transitions follow the Markov property.
- Each transition has a **probability** (transition matrix), telling how likely the system is to move between states.
- There are **no actions** and **no rewards** — just state transitions.
- They are often used to model random systems like weather, queues, or small simulations.
- Learning in this context focuses on estimating transition probabilities and understanding steady-state behaviour.
- Weakness: unable to model decisions or rewards — only passive evolution.

Markov Reward Process (MRP)

- An MRP is a Markov Chain **plus rewards**.
- Each state gives an immediate reward $R(s)$ and transitions are still Markovian.
- It also introduces **Return (\$G_t\$)** — the total discounted future reward.
- Used to evaluate how “good” each state is without worrying about actions or policies.

- MRPs are basically the “evaluation” backbone inside MDPs.
- Great for understanding value prediction before adding decisions.

Markov Decision Process (MDP)

- An MDP includes **states, actions, transition probabilities, rewards, and a discount factor**.
- It formally describes the RL environment.
- The agent chooses actions; the environment responds with new states and rewards.
- The goal is to find a **policy** that maximizes long-term return.
- MDPs allow reasoning about uncertainty, actions, and planning.
- They form the theoretical base for RL algorithms like Q-learning, PPO, SAC.

Important note:

Many learners confuse “MDP” with “Markov Property.”

But an MDP *uses* the Markov property — it’s not the same thing.

Return

- Return (G_t) is the **total future reward from time t onward**.
- Usually defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$
- γ (gamma) discounts future rewards to make the sum finite and prioritizes short-term outcomes.
- Return is used to define value functions, which drive policy evaluation.
- Mistake to avoid: thinking return is the same as reward — **reward is immediate; return is cumulative**.

Policy

- A policy tells the agent **what action to take in each state**.
- Can be deterministic (a specific action) or stochastic (a probability distribution).
- Policy learning aims to find the policy that maximizes expected returns.
- In model-free RL, policies are learned by interacting with the environment.
- In model-based RL, policies are learned by planning from a learned model.
- Good policies strike a balance between **exploration and exploitation**.

Value Functions

- Value functions estimate **how good a state or state-action pair is**.
- Two types:
 - **$V(s)$** : value of a state under a policy
 - **$Q(s,a)$** : value of taking action a in state s under a policy
- They help the agent plan long-term, beyond immediate rewards.
- Value functions smooth out randomness by predicting expected returns.
- Most RL algorithms learn these functions through sampling or bootstrapping.
- They can be represented as tables or neural networks (deep RL).

Bellman Equation

- The Bellman equation expresses the value of a state in terms of **immediate reward + value of the next state**.
- Example for state value:

$$V(s) = E[R + \gamma V(s')]$$
- It breaks down long-term prediction into smaller subproblems — a dynamic programming principle.
- It forms the backbone of value iteration, policy iteration, Q-learning, and deep RL.
- It simplifies learning by converting a huge prediction task into recursive steps.
- You should understand this equation well — examiners love it.

Introduction to Q-Learning

- Q-Learning is a **model-free reinforcement learning algorithm**. It learns the best actions **without needing a model** of the environment's transitions or rewards.
- It estimates **Q-values**, which measure how good it is to take a certain action in a certain state.
- The goal is to learn the **optimal policy** that maximizes long-term reward.
- It uses **trial-and-error**: the agent interacts with the environment, gets rewards, updates values.
- Q-Learning is **off-policy**, meaning it learns the optimal policy even if the agent follows an exploratory policy (like ϵ -greedy).
- Strengths: simple, effective on small discrete problems, converges under mild conditions.
- Weak point: does not scale well to large or continuous spaces without function approximators (Deep Q-Networks fix this).

Important Terms in Q-Learning

- **State (S)**: The situation the agent is currently in.
- **Action (A)**: The choice the agent can take from a state.
- **Q-Value (Q(s,a))**: Expected return of taking action a in state s .
- **Learning Rate (α)**: How much new information overrides old information.
- **Discount Factor (γ)**: How much the agent cares about future rewards.
- **Reward (R)**: Immediate feedback after an action.
- **Q-Table**: A table storing Q-values for every (state, action) pair.
- **Policy**: The rule that decides the next action (often ϵ -greedy for exploration).
- **Exploration vs Exploitation**: Choose new actions to learn vs use known good actions.
- **Temporal Difference (TD) Error**: The difference between expected and observed return — drives learning.

If any confusion: Q-value $\neq$ reward. Q-value is a long-term estimate.

Q-Table

- A Q-table is a **lookup table** storing $Q(s,a)$ for each state–action pair.
- Each row = a state; each column = an action.
- The agent updates table entries as it learns.
- Q-table is used only in environments with **discrete, small** state and action spaces.
- The table is updated using the Q-Learning update rule:
$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$
- Once trained, the best action in a state is simply the one with the highest Q-value.
- Mistake beginners make: thinking Q-table stores rewards — no, it stores *expected future returns*.

Q-Function

- The Q-function gives the **expected** discounted return for each (state, action) pair.
- Defined as:
$$Q\pi(s,a) = E[G_t \mid S_t = s, A_t = a, \text{ following policy } \pi]$$
- Q-Learning approximates the **optimal Q-function** Q^* by repeatedly updating Q-values.
- Once Q^* is learned, the optimal policy is:
$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$
- Q-function is the heart of Q-Learning; everything else builds around estimating it.
- Scales poorly for large spaces $\rightarrow$ in deep RL, neural networks approximate Q instead of using tables (DQN).

Q-Learning Algorithm

- Initialize the Q-table with zeros or small random values.

- Repeat for each episode:
 - Start in the initial state.
 - Choose an action using an exploration strategy (e.g., ϵ -greedy).
 - Perform the action, receive reward and next state.
 - Update Q-value using the Q-Learning update rule.
 - Move to the next state and continue.
- Continue training until Q-values converge.
- After training, the agent acts using the **greedy policy** (selecting best Q-values).
- This algorithm is **off-policy** because updates use $\max Q(s', a')$ — the greedy best action — not the action actually taken.

Callout:

If your Q-values are not converging, your α or ϵ schedule is probably wrong — fix those first.

3. Markov's Decision Process (MDP)

MDP is the **mathematical framework** used to describe Reinforcement Learning environments.

The Markov Property

The Markov Property states that: "The future depends only on the present, not on the past."

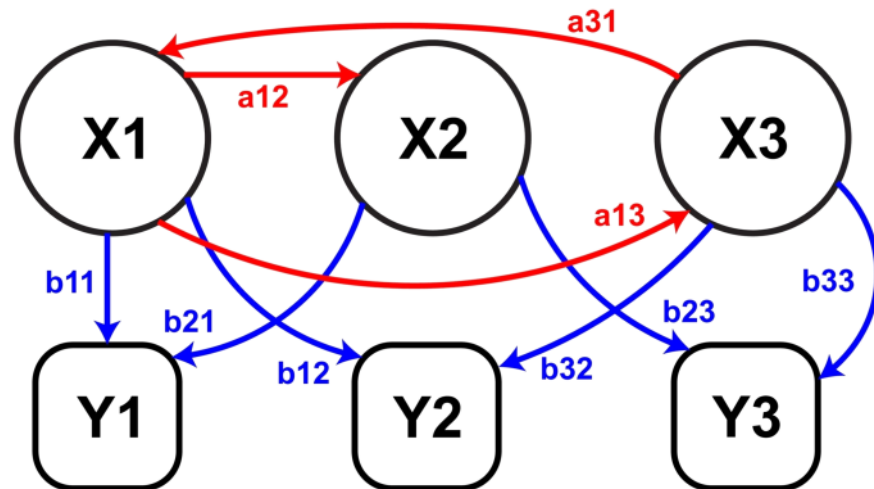
If we know the current state, the history of how we got there doesn't matter.

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_t, S_{t-1}, \dots, S_0)$$

1. Markov Chain (or Process)

A system that consists only of:

- A set of states (S).
- Transition Probabilities (P): The chance of moving from one state to another.
- *Note: There are no actions or rewards here. Just random movement.*



Hidden Markov model

Shutterstock

2. Markov Reward Process (MRP)

This is a Markov Chain **plus value**.

- **Tuple:** (S, P, R, γ)
- It adds **Rewards (R)** to the states. We can calculate how much "value" a state has.

3. Markov Decision Process (MDP)

This is the full RL framework. It adds **Actions**.

- **Tuple:** (S, A, P, R, γ)
 - S : Set of States.
 - A : Set of Actions.
 - P : Probability of transition.
 - R : Reward function.
 - γ (Γ): Discount factor.

Key Concepts in MDP

A. Return (G_t)

The total accumulated reward the agent expects to get from time t onwards.

Because the future is uncertain, we use a Discount Factor (γ) (between 0 and 1).

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- If $\gamma = 0$: The agent is "myopic" (cares only about immediate reward).
- If $\gamma = 1$: The agent is "farsighted" (cares about the distant future).

B. Policy (π)

A policy is the agent's "brain." It defines the behavior. It maps a State to an Action.

$$\pi(a|s) = P(A_t = a | S_t = s)$$

C. Value Functions

1. **State-Value Function $V(s)$** : How good is it to be in state s ? (Expected return starting from s).
2. **Action-Value Function $Q(s, a)$** : How good is it to take action a in state s ?

The Bellman Equation

The Bellman Equation breaks down the Value Function into two parts:

1. Immediate Reward.
2. Discounted value of the next state.

$$V(s) = \max_a [R(s, a) + \gamma V(s')]$$

This recursive equation allows us to solve MDPs iteratively.

4. Q-Learning

Introduction

Q-Learning is a **Model-Free, Off-Policy** Reinforcement Learning algorithm.

- **Model-Free**: It doesn't need to know the transition probabilities (P) of the environment beforehand.
- **Goal**: To learn the optimal policy by learning the **Q-values** (Quality) of action-state pairs.

Important Terms

- **Q-Value ($Q(s, a)$)**: The expected future reward for taking action a in state s .
- **Exploration**: Trying new random actions to discover new rewards.
- **Exploitation**: Choosing the known best action to maximize current rewards.
- **Epsilon-Greedy Strategy**: A method to balance exploration and exploitation. (e.g., 10% of the time explore, 90% of the time exploit).

The Q-Table

The Q-Table is a lookup table where:

- **Rows** = States
- **Columns** = Actions
- **Cell Value** = The calculated Q-value (maximum expected future reward).

Initially, all values in the Q-table are initialized to 0. As the agent explores, these numbers are updated.

The Q-Learning Algorithm (Steps)

1. **Initialize**: Create the Q-Table with all zeros.
2. **Select Action**: Using the Epsilon-Greedy policy:
 - Select a random action (Explore) OR
 - Select the action with the highest Q-value in the current row (Exploit).
3. **Perform Action**: Take the action a , reach next state s' , and receive reward r .
4. **Update Q-Value**: Use the **Bellman Update Equation** to update the Q-table value for the previous state.
5. **Repeat**: Move to the next state and repeat until the episode ends.

Q-Learning Update Function (The Equation)

This is the most critical formula to memorize for the exam:

$$Q(s, a)_{\text{new}} = Q(s, a)_{\text{old}} + \alpha [R + \gamma \max_{a'} (Q(s', a')) - Q(s, a)_{\text{old}}]$$

Where:

- α (Alpha): **Learning Rate** (How much we accept new information vs old information).

- γ (Gamma): **Discount Factor**.
- R : Reward received.
- $\max_a Q(s, a)$: The maximum predicted reward for the *next* state (best possible future).

From <<https://gemini.google.com/app/85dc9ef8bd027daf>>

Classification

12 December 2025 08:53

1. Introduction: Need of Classification

What is Classification?

Classification is a **supervised machine learning task** where the goal is to assign input data to one of several predefined categories or classes.

Example:

- Email → *spam / not spam*
- Image → *cat, dog, car*
- Medical data → *disease present / disease absent*

Why is Classification Needed?

Classification is needed because many real-world problems require **decision-making based on categories**, not continuous values.

1. Automation of Decisions

Machines can categorize data automatically.

Examples:

- Fraud detection by classifying transactions as *fraudulent* or *legitimate*
- Document sorting into *topics*

2. Handling Large-Scale Data

Manual classification is impossible for millions of data points (emails, images, sensors, etc.)

3. Predictive Capability

Classification helps predict the category of unseen data.

4. Pattern Recognition

Allows systems to recognize:

- Faces
- Speech
- Handwriting
- Anomalies (abnormal patterns)

5. Risk and Medical Diagnosis

Classification helps assess:

- Risk level (high / low)
- Disease prediction
- Customer churn

2. Types of Classification

There are two primary types:

(A) Binary Classification

Here, the data is classified into **two classes**.

Examples:

- Spam vs Not spam
- Healthy vs Diseased
- Pass vs Fail
- Positive vs Negative

The classes are usually labeled as **0 and 1**, or **-1 and +1**.

Characteristics

- Simple decision boundary
- Easier to model
- Many algorithms are originally designed for binary cases (e.g., SVM, Logistic Regression)

(B) Multiclass Classification

Multiple categories (>2) exist.

Examples:

- Handwritten digits (0–9) → 10 classes
- Animal classification (dog, cat, horse, cow)
- Weather (sunny, cloudy, rainy, snowy)

Characteristics

- More complex decision boundaries
- Requires different strategies since many algorithms are inherently binary

3. Binary vs Multiclass Classification

Binary Classification

- Only **two** classes
- Simpler, faster
- Many algorithms work naturally
- Less computational cost
- Evaluation metrics (accuracy, precision, recall) are straightforward

Multiclass Classification

- More than two classes
- Requires:
 - **One-vs-One (OvO)**
 - **One-vs-Rest (OvR)**
 - or dedicated multiclass algorithms (e.g., Random Forest, Neural Networks)
- More training time
- Evaluation becomes more complex (macro, micro averaging)

Differences (Exam-friendly Table)

Feature	Binary Classification	Multiclass Classification
Number of classes	2	3 or more
Model complexity	Low	High
Training time	Faster	Slower
Examples	Spam detection	Digit recognition
Handling	Direct	Requires OvO or OvR strategies
Evaluation	Simple	Uses averaging (macro/micro F1)

4. Balanced vs Imbalanced Classification Problems

Balanced Classification Problem

A dataset is **balanced** when each class has approximately the **same number of samples**.

Example:

- 500 positive cases
 - 520 negative cases
- Difference is small → balanced.

Characteristics

- Classifiers perform well
- Accuracy is reliable
- No need for special techniques

Imbalanced Classification Problem

A dataset is **imbalanced** when one class has far more samples than the other.

Example:

- Fraud detection:
 - Normal transactions = 99,000
 - Fraud = 1,000
- Medical rare disease detection
- Machine failure detection

These cases are common in real-life.

Problems Caused by Imbalance

- Model becomes biased toward majority class
- High accuracy but poor performance on minority class
- Example:
 - Predicting “non-fraud” all the time gives **99% accuracy** but is useless

Solutions

- **Resampling methods**
 - Oversampling minority (SMOTE)
 - Undersampling majority
- **Class weighting**
 - Give more penalty for misclassifying the minority class
- **Anomaly detection models**
 - For extremely rare events
- **F1-score instead of accuracy** as metric

1. Binary Classification

Binary classification involves predicting one of **two classes**.

Examples:

- Spam vs Not spam
- Positive vs Negative
- Fraud vs Normal

The output is usually **0** or **1**.

2. Linear Classification Model

A **linear classifier** predicts the class using a **linear decision boundary** (a straight line or hyperplane).

General Form

$$f(x) = wTx + b \quad \text{if } w^T x + b \geq 0 \\ f(x) = -wTx + b \quad \text{if } w^T x + b < 0$$

Decision Rule

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ -1 & \text{if } w^T x + b < 0 \end{cases}$$

Examples of Linear Classifiers

- Logistic Regression
- Linear SVM
- Perceptron
- Linear Discriminant Analysis

Geometric Interpretation

- The vector **w** defines the orientation of the decision boundary.
- **b** shifts the boundary.

Advantages

- Fast to train
- Easy to interpret
- Works well for high-dimensional data (e.g., text classification)

Limitations

- Cannot handle complex non-linear boundaries
- Often need feature transformations or kernels

3. Performance Evaluation: Confusion Matrix

A **confusion matrix** summarizes classifier performance by comparing **predicted vs actual labels**.

Actual \ Predicted	Positive (1)	Negative (0)
Positive (1)	TP (True Positive)	FN (False Negative)
Negative (0)	FP (False Positive)	TN (True Negative)

Definitions

- **TP**: Predicted positive and actual positive
- **TN**: Predicted negative and actual negative
- **FP**: Predicted positive but actual negative
- **FN**: Predicted negative but actual positive

Confusion matrix is the basis for all evaluation metrics.

4. Accuracy

Accuracy measures the **overall correctness** of the classifier.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

When Accuracy is Good

- When classes are **balanced**

When Accuracy Fails

- In **imbalanced datasets**
Example:
 - Fraud = 1%
 - Normal = 99%Predicting “normal” always → 99% accuracy but useless

5. Precision

Precision measures **how many predicted positives are actually positive**.

$$\text{Precision} = \frac{TP}{TP + FP}$$

High Precision means:

- Very few **false positives**

Used when:

- FP must be minimized
Examples:
 - Spam detection (don't mark good emails as spam)
 - Disease diagnosis (avoid false alarms)

6. Recall

Recall (Sensitivity / True Positive Rate) measures **how many actual positives are correctly detected**.

$$\text{Recall} = \frac{TP}{TP + FN}$$

High Recall means:

- Very few **false negatives**

Used when:

- FN must be minimized
Examples:
 - Cancer detection (catch all cases)

- Fraud detection (don't miss frauds)

7. F-measures (F1 Score)

The **F-measure** combines both Precision and Recall into a single metric.

F1 Score (most common)

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

It is the **harmonic mean** of Precision and Recall.

Why Harmonic Mean?

It punishes extreme values.

Example:

- Precision = 1
 - Recall = 0
- F1 = 0 → even though precision is perfect.

When F1 is useful

- Imbalanced datasets
- When both precision and recall matter

1. Multiclass Classification

Multiclass classification deals with predicting one class out of **three or more categories**.

Examples:

- Digit recognition: 0–9 (10 classes)
- Animal classification: cat, dog, horse, cow
- Sentiment: positive, neutral, negative

Most machine learning algorithms are inherently **binary**, so special strategies are used for multiclass classification.

2. One-vs-One (OvO) Classification Technique

Concept

For **K classes**, train a **separate classifier for every pair of classes**.

Total classifiers required:

$$\frac{K(K-1)}{2}$$

How it works

1. Build a classifier for each pair (e.g., Class 1 vs Class 2)
2. Each classifier predicts one of the two classes
3. During prediction:
 - Each classifier votes for a class
 - The class with **maximum votes** is chosen

Example (3 classes: A, B, C)

Classifiers trained:

- A vs B
- A vs C
- B vs C

Advantages

- Works very well for **SVM**
- Each classifier deals with fewer samples (only two classes each)
- Good accuracy

Disadvantages

- High number of models for large K
- More training time

3. One-vs-All (One-vs-Rest, OvR) Classification Technique

Concept

For **K classes**, train **K** binary classifiers.

Each classifier separates:

- One class vs
- All other classes combined

Example (3 classes: A, B, C)

You train:

- Classifier 1: A vs not-A
- Classifier 2: B vs not-B
- Classifier 3: C vs not-C

Prediction

- Compute decision score for each classifier
- Choose the class with **highest score**

Advantages

- Only K models (much fewer than OvO)
- Simpler and faster
- Works well for logistic regression and linear classifiers

Disadvantages

- Biased when classes overlap heavily
- Class imbalance in each classifier (one class vs many)

4. Multiclass Confusion Matrix

In multiclass problems, the confusion matrix becomes an **N × N** table (where N = number of classes).

Rows → Actual class

Columns → Predicted class

Example: 3-class confusion matrix

Actual \ Predicted	A	B	C
A	TPA	F(A→B)	F(A→C)
B	F(B→A)	TPB	F(B→C)
C	F(C→A)	F(C→B)	TPC

Interpretation

- Diagonal elements = correctly classified (TP for each class)
- Off-diagonal elements = misclassifications

5. Per-Class Precision

Precision is computed **for each class separately**.

For class i:

$$\text{Precision}_i = \frac{\text{True Positives for class } i}{\text{Predicted as class } i} = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}$$

Formula

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}$$

Where:

- **TP_i** = correctly predicted class i
- **FP_i** = predicted class i but belongs to other class

Meaning

“How accurate are predictions for class i?”

When useful

- When **false positives** are important
- Example: predicting "Cancer" class incorrectly should be avoided

6. Per-Class Recall

Recall is also computed **individually for each class**.

$$\text{Recall}_i = \frac{\text{True Positives for class } i}{\text{Actual instances of class } i}$$

= Actual instances of class i / True Positives for class i

Formula

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

Where:

- **TP_i** = correct predictions for class i
- **FN_i** = instances of class i predicted as another class

Meaning

“How many actual class-i samples were detected correctly?”

When useful

- When **false negatives** are important
- Example: failing to detect “fraud” (FN) is costly

Macro, Micro, and Weighted (Optional but exam-relevant)

In multiclass problems, you may combine per-class precision/recall:

Macro Average

- Computes metric for each class
- Takes simple average
- Treats all classes equally

Micro Average

- Aggregate TP, FP, FN for all classes
- Best when class imbalance exists

Weighted Average

- Weighted by class population
- Most useful for imbalanced multiclass datasets

1. Linear Support Vector Machines (SVM) – Introduction

What is an SVM?

A Support Vector Machine is a **supervised learning algorithm** used for **binary classification** and regression.

Its goal: **find the optimal separating hyperplane** that maximizes the margin between two classes.

Key Terminology

- **Hyperplane:** A decision boundary that separates classes.
 - In 2D → a line
 - In 3D → a plane
 - In n-dimensions → (n–1)-dimensional surface
- **Margin:** Distance between the hyperplane and the nearest data points.
- **Support Vectors:** Data points that lie closest to the hyperplane and define it.

Objective of Linear SVM

Maximize the margin:

$$\max_{w,b} \frac{2}{\|\mathbf{w}\|} \quad \text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i$$

Subject to the constraint:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i$$

2. Soft Margin SVM

Why Soft Margin?

Real-world data is rarely perfectly separable.

Soft-margin SVM allows:

- Misclassified points
- Points inside the margin

This is done using **slack variables** ξ_i .

Optimization function

$$\min_{w,b,\xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Subject to:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Role of C (Regularization Parameter)

- **High C:** Low tolerance to misclassification → narrow margin → risk of overfitting
- **Low C:** High tolerance → wider margin → risk of underfitting

3. Kernel Functions

Why Kernels?

Some data is **not linearly separable** in the original space.

Kernel trick maps data into a **higher-dimensional space** without explicitly computing the transformation.

General formula:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}') \quad K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$$

4. Radial Basis Function (RBF) Kernel

Formula

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2) \quad K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

Parameters

- **γ (gamma):**
 - High gamma → tighter influence → more complex boundary (may overfit)
 - Low gamma → smoother boundary

Properties

- Very popular for non-linear classification
- Able to model complex shapes
- Infinite-dimensional feature space

5. Gaussian Kernel Learning

Gaussian kernel is **same as RBF kernel**.

Named because of the Gaussian distribution shape.

Why it works well?

- Local influence around data points
- Smooth decision boundary

- Can separate data by forming "bubbles" around points

6. Polynomial Kernel

Formula

$$K(x, x') = (x \cdot x' + c)^d$$

Parameters

- **d**: degree of the polynomial
- **c**: constant (usually 1)

Properties

- Allows curved decision boundaries
- Low degree (2–3) works best
- High degree → overfitting

Use Cases

- Text classification
- Bioinformatics
- When relationships between features are polynomial in nature

7. Sigmoid Kernel

Formula

$$K(x, x') = \tanh(\alpha x \cdot x' + c)$$

Parameters

- **α**: scale parameter
- **c**: offset parameter

Properties

- Similar to hidden layer neurons in neural networks
- Not always a valid Mercer kernel (depends on α and c)
- Less widely used than RBF or polynomial

K-Nearest Neighbour (KNN) Algorithm – Detailed Note

K-Nearest Neighbour (KNN) is one of the **simplest and most intuitive supervised machine learning algorithms** used for both **classification and regression**. It is a **lazy learning, instance-based, and non-parametric** method.

1. Introduction to KNN

KNN works on the principle that:

“Similar data points exist close to each other in feature space.”

So, to classify a new point, KNN looks at the **K nearest labelled data points** and assigns the class that is most common (majority vote).

Key Characteristics

- **Lazy learner**: No model is built during training; all computation happens during prediction.
- **Non-parametric**: Makes no assumption about data distribution.
- **Distance-based**: Uses distance metrics like Euclidean, Manhattan, Minkowski.

2. How KNN Works (Step-by-Step)

Step 1: Choose the value of K

- K = number of nearest neighbors to consider
- Common values: 3, 5, 7
- Small K → noisy, overfitting
- Large K → smooth, may underfit

Step 2: Calculate Distance

For each training point, compute distance from the new input.

Most common: Euclidean distance

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Other distances:

- Manhattan
- Minkowski
- Hamming distance (for categorical data)

Step 3: Find K Nearest Neighbours

Sort all distances and pick the **K** smallest.

Step 4: Perform Voting

- **Classification:** choose class with majority votes
- **Regression:** take average of K values

Step 5: Assign the Class or Output

Assign the label that wins the voting.

3. Example of KNN Classification (Very Important for Exams)

Dataset (2 features: Height, Weight)

Person	Height (cm)	Weight (kg)	Class
A	170	65	Fit
B	160	60	Fit
C	175	80	Unfit
D	180	85	Unfit

New Person (X):

Height = 172, Weight = 70 → Predict Fit / Unfit

Let **K = 3**

Step 1: Calculate Euclidean distance

$$d = \sqrt{(h_1 - h_2)^2 + (w_1 - w_2)^2}$$

Training Point	Distance to X	Class
A (170,65)	$\sqrt{((172-170)^2 + (70-65)^2)} = \sqrt{(4+25)} = \sqrt{29} = 5.38$	Fit
B (160,60)	$\sqrt{((172-160)^2 + (70-60)^2)} = \sqrt{(144+100)} = \sqrt{244} = 15.62$	Fit
C (175,80)	$\sqrt{((172-175)^2 + (70-80)^2)} = \sqrt{(9+100)} = \sqrt{109} = 10.44$	Unfit
D (180,85)	$\sqrt{((172-180)^2 + (70-85)^2)} = \sqrt{(64+225)} = \sqrt{289} = 17$	Unfit

Step 2: Pick the 3 Nearest Neighbours

Nearest 3 distances:

- A → 5.38 (Fit)
- C → 10.44 (Unfit)
- B → 15.62 (Fit)

Step 3: Voting

Fit = 2
Unfit = 1

Final Prediction

The new person is predicted as **Fit**.
The new person is predicted as **Fit**.

4. Choosing the Best Value of K

- **Odd values** preferred for binary classification to avoid ties
- **Small K**: very sensitive to noise
- **Large K**: smoother boundary but may misclassify minority classes
- Typically choose K using **cross-validation**

5. Advantages of KNN

✓ Simple and intuitive

Easy to understand and implement.

✓ No training time

Suitable when training dataset changes frequently.

✓ Flexible

Can be used for:

- classification
- regression
- multi-class tasks

✓ Works well on small datasets

Good for memory-based decision systems.

6. Disadvantages of KNN

✗ Slow prediction

Because distance to all training points must be computed.

✗ High memory usage

Stores all training data.

✗ Sensitive to scale

Features with larger values dominate distance.

🔧 Solution: Use normalization/standardization.

✗ Sensitive to irrelevant features

Feature selection is important.

✗ Not good for very large datasets

Requires optimizations like KD-tree, Ball-tree.

7. Applications of KNN

- Recommender systems
- Pattern recognition
- Image classification
- Medical diagnosis
- Credit scoring
- Anomaly detection